MBU DIGITAL LIBRARY · PRACTICE LEARNING RESOURCE

C Programming

CS101 | Suggested sequence · Semester 1 | Student-authored study guide

How to use these notes

C fundamentals, functions, arrays, pointers, structures, and memory-safe problem solving. Work through the units in order, reproduce each example, then answer the review prompts from memory. This is supplemental practice material, not an official MBU syllabus or assessment. Course sequencing is suggested and should be checked against your current approved syllabus.

MODULE 01 · Types, operators, and control flow

C is statically typed. Include the right headers, use format specifiers that match types, and check input conversions. Integer division discards the fractional part. Use braces consistently, even for one statement.

Worked example

```
int value; if (scanf("%d", &value) != 1) { return 1; }
```

Check your understanding

- Why does scanf need & for an int variable?
- What can happen when signed integer overflow occurs?

MODULE 02 · Functions and arrays

Arrays contain contiguous elements and do not carry a runtime length. Pass a length alongside an array. A function declaration communicates its parameter and return types. Keep each function focused and avoid relying on global mutable state.

Worked example

```
int sum(const int values[], size_t count) { int total = 0; for (size_t i = 0; i < count; i++)
total += values[i]; return total; }
```

Check your understanding

- Why pass array length as a parameter?
- What is the valid last index of an array of length n?

MODULE 03 · Pointers, structures, and memory

A pointer stores an address. Initialize pointers, check allocation results, free each successful allocation exactly once, and never dereference a dangling pointer. Structures group related fields; use size_t for object sizes.

Worked example

```
int *items = malloc(count * sizeof *items); if (items == NULL) { return 1; } /* use items */
free(items);
```

Check your understanding

- What is a dangling pointer?

• Why is sizeof *items safer than repeating the type?

Revision checklist

- %¡ Can I explain the main idea without copying the notes?
- %¡ Can I reproduce the example and state its assumptions?
- %¡ Can I test an edge case and describe what should happen?
- %¡ Have I recorded a limitation and checked a primary reference?

Further reading

Use the official documentation and open references linked in the portal. Check the publication/version and license at the source before redistribution. Cite the source and access date in coursework.